

Q1. Smart Hospital Patient Registration System

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.*;

class Main {
    static ArrayList<Patient> patientlist = new ArrayList<>();
    static HashMap<String, Patient> patientMap = new HashMap<>();

    public static String readNext(Scanner sc) {
        // Reads next token OR next line if tokenizing breaks names
        // Here we want full lines for names/doctors, so we read lines.
        while (sc.hasNextLine()) {
            String s = sc.nextLine();
            if (s == null) return null;
            s = s.trim();
            if (!s.isEmpty()) return s;
        }
        return null;
    }

    public static String properCase(String name) {
        name = name.trim().replaceAll("\\s+", " ");
        if (name.isEmpty()) return "";
        name = name.toLowerCase();
        String[] words = name.split(" ");
        StringBuilder sb = new StringBuilder();
        for (String w : words) {
            if (w.isEmpty()) continue;
            sb.append(Character.toUpperCase(w.charAt(0)));
            if (w.length() > 1) sb.append(w.substring(1));
            sb.append(" ");
        }
        return sb.toString().trim();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            String choiceStr = readNext(sc);
            if (choiceStr == null) break;

            int choice;
            try {
                choice = Integer.parseInt(choiceStr);
            } catch (Exception e) {
                System.out.println("Invalid Choice");
                continue;
            }

            switch (choice) {
                case 1: {
                    if (patientlist.size() >= 100) {
                        System.out.println("Maximum Patient Limit Reached");
                        break;
                    }

                    String id = readNext(sc);
                    if (id == null || id.trim().isEmpty()) {
                        System.out.println("Invalid Patient ID");
                        break;
                    }
                }
            }
        }
    }
}
```

```
id = id.trim();

if (patientMap.containsKey(id)) {
    System.out.println("Duplicate Patient ID");
    break;
}

String rawName = readNext(sc);
if (rawName == null) { System.out.println("Invalid Patient Name"); break; }
String name = properCase(rawName);

if (name.isEmpty() || !name.matches("[A-Za-z ]+")) {
    System.out.println("Invalid Patient Name");
    break;
}

String mobile = readNext(sc);
if (mobile == null) { System.out.println("Invalid Mobile Number"); break; }
mobile = mobile.trim();
if (!mobile.matches("\\d{10}")) {
    System.out.println("Invalid Mobile Number");
    break;
}

String department = readNext(sc);
if (department == null) { System.out.println("Invalid Department"); break; }
department = department.trim().replaceAll("\\s+", " ").toUpperCase();

String doctor = readNext(sc);
if (doctor == null) { System.out.println("Invalid Doctor Name"); break; }
doctor = doctor.trim().replaceAll("\\s+", " ");

Patient p = new Patient(id, name, mobile, department, doctor);
patientlist.add(p);
patientMap.put(id, p);

System.out.println("Patient Registered Successfully.");
break;
}

case 2: {
    String searchId = readNext(sc);
    if (searchId != null) searchId = searchId.trim();

    if (searchId != null && patientMap.containsKey(searchId)) {
        Patient p = patientMap.get(searchId);
        System.out.println("Patient Details");
        System.out.println("ID : " + p.getPatientId());
        System.out.println("Name : " + p.getPatientName());
        System.out.println("Mobile : " + p.getMobileNumber());
        System.out.println("Department : " + p.getDepartment());
        System.out.println("Doctor : " + p.getDoctorName());
    } else {
        System.out.println("Patient Not Found");
    }
    break;
}

case 3: {
    System.out.println("-----");
    System.out.println("ID Name Mobile Department Doctor");
    System.out.println("-----");
    for (Patient p : patientlist) {
        System.out.println(
```

```
                p.getPatientId() + " " +
                p.getPatientName() + " " +
                p.getMobileNumber() + " " +
                p.getDepartment() + " " +
                p.getDoctorName()
            );
        }
        break;
    }

    case 4: {
        // Preserve first-seen department order
        LinkedHashMap<String, Integer> countMap = new LinkedHashMap<>();
        for (Patient p : patientList) {
            String dept = p.getDepartment();
            countMap.put(dept, countMap.getOrDefault(dept, 0) + 1);
        }

        System.out.println("Department Count");
        for (Map.Entry<String, Integer> e : countMap.entrySet()) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
        break;
    }

    case 5:
        sc.close();
        return;

    default:
        System.out.println("Invalid Choice");
    }
}
sc.close();
}

class Patient {
    private String patientId;
    private String patientName;
    private String mobileNumber;
    private String department;
    private String doctorName;

    public Patient(String patientId, String patientName, String mobileNumber,
                    String department, String doctorName) {
        this.patientId = patientId;
        this.patientName = patientName;
        this.mobileNumber = mobileNumber;
        this.department = department;
        this.doctorName = doctorName;
    }

    public String getPatientId() { return patientId; }
    public String getPatientName() { return patientName; }
    public String getMobileNumber() { return mobileNumber; }
    public String getDepartment() { return department; }
    public String getDoctorName() { return doctorName; }
}
```

INPUT

```
1
P101
Rahul Kumar
9876543210
Cardiology
```



```
Dr.Arun
1
P102
Priya
9876543211
Ortho
Dr.Meena
3
4
2
P101
5
6
```

OUTPUT

```
Patient Registered Successfully.
Patient Registered Successfully.
-----
ID Name Mobile Department Doctor
-----
P101 Rahul Kumar 9876543210 CARDIOLOGY Dr.Arun
P102 Priya 9876543211 ORTHO Dr.Meena
Department Count
CARDIOLOGY : 1
ORTHO : 1
Patient Details
ID : P101
Name : Rahul Kumar
Mobile : 9876543210
Department : CARDIOLOGY
Doctor : Dr.Arun
```

Q2. Online Library Book Management System

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.*;

class Main {

    static class Book {
        String bookId;
        String title;
        String author;
        String category;
        String publisher;

        Book(String bookId, String title, String author, String category, String publisher) {
            this.bookId = bookId;
            this.title = title;
            this.author = author;
            this.category = category;
            this.publisher = publisher;
        }
    }

    // Read non-empty line (drives menu and fields safely)
    static String readInput(Scanner sc) {
        while (sc.hasNextLine()) {
            String s = sc.nextLine();
            if (s == null) return "";
            s = s.trim();
            if (!s.isEmpty()) return s;
        }
        return "";
    }

    // Title Case conversion + space normalization
    static String toTitleCase(String s) {
        s = s.trim().replaceAll("\\s+", " ");
        if (s.isEmpty()) return "";

        s = s.toLowerCase();
        String[] parts = s.split(" ");

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < parts.length; i++) {
            String w = parts[i];
            if (w.isEmpty()) continue;
            sb.append(Character.toUpperCase(w.charAt(0)));
            if (w.length() > 1) sb.append(w.substring(1));
            if (i < parts.length - 1) sb.append(" ");
        }
        return sb.toString().trim();
    }

    static String normalizeSpaces(String s) {
        return s.trim().replaceAll("\\s+", " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Book> books = new ArrayList<>();
        HashMap<String, Book> bookMap = new HashMap<>(); // Book ID -> Book
```

```
// Menu loop
while (true) {
    String choiceStr = readInput(sc);
    if (choiceStr.isEmpty()) break;

    int choice;
    try {
        choice = Integer.parseInt(choiceStr);
    } catch (NumberFormatException e) {
        System.out.println("Invalid Choice");
        continue;
    }

    switch (choice) {
        case 1: { // Add Book
            if (books.size() >= 500) {
                System.out.println("Maximum Limit Reached");
                break;
            }

            String bookId = normalizeSpaces(readInput(sc));
            if (bookId.isEmpty()) {
                System.out.println("Book ID Cannot be Empty");
                break;
            }

            if (bookMap.containsKey(bookId)) {
                System.out.println("Duplicate Book ID");
                break;
            }

            String rawTitle = readInput(sc);
            String title = toTitleCase(rawTitle);

            String author = toTitleCase(readInput(sc)); // best match for "String handling"
            String category = normalizeSpaces(readInput(sc)).toUpperCase();
            String publisher = normalizeSpaces(readInput(sc));

            if (publisher.isEmpty()) {
                System.out.println("Publisher Cannot be Empty");
                break;
            }

            Book b = new Book(bookId, title, author, category, publisher);
            books.add(b);
            bookMap.put(bookId, b);

            System.out.println("Book Added Successfully.");
            break;
        }

        case 2: { // Search Book by ID
            String searchId = normalizeSpaces(readInput(sc));

            if (bookMap.containsKey(searchId)) {
                Book b = bookMap.get(searchId);
                System.out.println(b.bookId);
                System.out.println(b.title);
                System.out.println(b.author);
                System.out.println(b.category);
                System.out.println(b.publisher);
            } else {
                System.out.println("Book Not Available");
            }
        }
    }
}
```

```
        break;
    }

    case 3: { // Display All Books
        System.out.println("-----");
        System.out.println("ID TITLE AUTHOR CATEGORY");
        System.out.println("-----");

        for (Book b : books) {
            System.out.println(
                b.bookId + " " +
                b.title + " " +
                b.author + " " +
                b.category
            );
        }
        break;
    }

    case 4: { // Display Unique Authors
        HashSet<String> authorSet = new HashSet<>();
        for (Book b : books) authorSet.add(b.author);

        System.out.println();
        System.out.println("Unique Authors");
        // deterministic order (helps with test cases)
        ArrayList<String> authors = new ArrayList<>(authorSet);
        Collections.sort(authors);

        for (String a : authors) {
            System.out.println(a);
        }
        break;
    }

    case 5: { // Category-wise Book Count + String Operations (Requirement 6)
        HashMap<String, Integer> countMap = new HashMap<>();
        for (Book b : books) {
            countMap.put(b.category, countMap.getOrDefault(b.category, 0) + 1);
        }

        // Category-wise Count
        System.out.println("Category-wise Count");
        ArrayList<String> cats = new ArrayList<>(countMap.keySet());
        Collections.sort(cats);

        for (String c : cats) {
            System.out.println(c + " : " + countMap.get(c));
        }

        // String operations for each book
        System.out.println();
        // If they want the sample-style output: print details with length/words/category.
        // We'll follow the spirit of requirement 6:
        for (Book b : books) {
            String title = b.title;
            int length = title.length();
            int words = title.isEmpty() ? 0 : title.trim().split("\\s+").length;

            System.out.println("Book Details");
            System.out.println("Book ID : " + b.bookId);
            System.out.println("Title : " + b.title);
            System.out.println("Author : " + b.author);
            System.out.println("Category : " + b.category);
        }
    }
}
```



```
        System.out.println("Publisher : " + b.publisher);

        // extra required string outputs
        System.out.println("Length : " + length);
        System.out.println("Words : " + words);
        System.out.println();
    }
    break;
}

case 6: { // Exit
    sc.close();
    return;
}

default:
    System.out.println("Invalid Choice");
}
}

sc.close();
}
```

INPUT

```
1
B101
Java Programming
James Gosling
Programming
Pearson
1
B102
Data Structures
Mark Allen
Computer Science
McGraw Hill
2
B101
3
4
5
6
```

OUTPUT

```
Book Added Successfully.
Book Added Successfully.
B101
Java Programming
James Gosling
PROGRAMMING
Pearson
-----
ID TITLE AUTHOR CATEGORY
-----
B101 Java Programming James Gosling PROGRAMMING
B102 Data Structures Mark Allen COMPUTER SCIENCE

Unique Authors
James Gosling
Mark Allen
Category-wise Count
COMPUTER SCIENCE : 1
PROGRAMMING : 1

Book Details
Book ID : B101
Title : Java Programming
Author : James Gosling
Category : PROGRAMMING
Publisher : Pearson
Length : 16
Words : 2

Book Details
Book ID : B102
Title : Data Structures
Author : Mark Allen
Category : COMPUTER SCIENCE
```

Publisher : McGraw Hill
Length : 15
Words : 2